

Here is your complete discovery report for **Knowing Where to Start**. It is formatted in clean, mobile-friendly Markdown so you can copy and paste it directly into a file like `Code_Exploration_Report.md` in your GitHub repository or into a document.

Codebase Exploration & Implementation Report

Project: Task Manager Application (Python 3.11)

Document Title: Knowing Where to Start — Codebase Discovery

Author: Software Engineering Team Member

Part 1: Project Structure & Architectural Analysis

Initial Impressions vs. Codebase Reality

- **Initial Assumption:** Expected third-party frameworks, database libraries, and dependency manifests like `requirements.txt` or `Pipfile`.
- **Actual Discovery:** Built entirely on Python 3.11's standard library (`json`, `argparse`, `datetime`, `unittest`). No external dependencies are required.

Core Architecture & Entry Points

The application uses a clean **4-Layered Architecture**:

1. **Presentation Layer (`cli.py`):** The primary entry point. Parses CLI arguments using `argparse` and routes commands to the service layer.
2. **Service Layer (`task_manager.py`):** Houses the main business rules, coordinate task queries, updates status, and filters tasks.
3. **Domain Layer (`models.py`):** Defines core entities (`Task`) and Enums (`TaskStatus`, `TaskPriority`).
4. **Data Access Layer (`repository.py`):** Handles file persistence, reading and writing JSON payloads to `tasks.json`.

Part 2: Feature Implementation Location (Task Export to CSV)

Search Strategy & Findings

- **Search Terms Used:** `save`, `file`, `json`, `dump`, `write`.
- **Existing Pattern Found:** Data transformation and file writing currently live in `repository.py` (handling JSON serialization via `json.dump`).

Implementation Plan for "CSV Export"

To keep code modular without breaking existing layers:

1. **New Exporter Utility (`csv_exporter.py`):** Create a dedicated helper using Python's standard `csv` module to handle converting `Task` objects into CSV formatting.

2. **Service Layer (task_manager.py):** Add an `export_to_csv(filepath)` method that retrieves tasks from the repository and passes them to the CSV exporter.
3. **CLI Layer (cli.py):** Add a new command option: `python cli.py export --format csv --output tasks.csv`.

Part 3: Domain Model & Business Concepts

Core Entities & Relationships

- **Task Entity:** Represents individual work items. Contains metadata including `id`, `title`, `description`, `status`, `priority`, `due_date`, `tags`, and `created_at`.
- **TaskStatus Enum:** Defines allowable lifecycle states:
 - `TODO`
 - `IN_PROGRESS`
 - `REVIEW`
 - `DONE`
- **TaskPriority Enum:** Mapped numerical priority levels:
 - `1 = LOW`
 - `2 = MEDIUM`
 - `3 = HIGH`
 - `4 = URGENT`

Domain Concept Glossary

- **Overdue Task:** Any task where `due_date < current_date` and `status != TaskStatus.DONE`.
- **Priority Escalation:** Numerical comparison where higher numbers take operational precedence.

Part 4: Practical Application — Overdue Rule

Implementation

Scenario

Rule: Tasks overdue for more than 7 days should automatically be marked as abandoned, **unless** they are marked as high priority (HIGH or URGENT).

Modification Plan

1. Models (models.py)

- Extend the `TaskStatus Enum` to include an `ABANDONED` status.

2. Service Layer (task_manager.py)

Add a new method `process_overdue_tasks()`:

- Retrieve all active tasks (`status != DONE` and `status != ABANDONED`).
- Filter for tasks where `(current_date - task.due_date).days > 7`.

- Exclude tasks where `task.priority >= TaskPriority.HIGH` (protecting priority levels 3 and 4).
- Set remaining tasks to `TaskStatus.ABANDONED` and trigger `repository.save_all()`.

3. Presentation Layer (`cli.py`)

- Add a flag or command: `python cli.py cleanup-overdue` (or automatically run this check during `python cli.py list`).

4. Automated Tests (`tests/test_task_manager.py`)

- Add unit test cases verifying that tasks overdue by 8 days are marked `ABANDONED` if priority is `LOW` or `MEDIUM`, but remain untouched if priority is `HIGH` or `URGENT`.

Clarification Questions for the Team

1. Should `ABANDONED` tasks still appear in general list commands, or be hidden by default?
2. Should the system log or notify users when a task is automatically marked as `ABANDONED`?

Final Summary & Personal Reflection

Initial vs. Final Understanding

Initially, navigating unfamiliar code without running an IDE or debugger felt overwhelming. By analyzing entry points (`cli.py`), domain models (`models.py`), and storage routines (`repository.py`), the codebase transformed from a collection of loose files into a simple, logical pipeline.

Value of AI Prompt Strategies

- **Structure Prompts:** Revealed architectural patterns (Layered / Repository patterns) without getting bogged down in syntax.
- **Feature Location Prompts:** Saved time tracing file logic by isolating exactly where persistence and data transformation belong.
- **Domain Prompts:** Clarified data constraints (like Enum mappings) to prevent bad assumptions about how variables behave.

Future Code Strategy

When approaching any new codebase:

1. Start at entry points and execution interfaces (CLI, API routes, or main scripts).
2. Trace where data enters, transforms, and persists.
3. Map out core entities and status Enums before attempting to write or modify code.